Session 10: Structures & Unions (C Language)

A. Structures in C

1. What is a Structure?

A **structure** in C is a **user-defined data type** that allows us to group **different types of variables** under a single name.

👉 Structures are used when we want to represent a **real-world entity** that has multiple properties.

Why we need structures?

- Arrays store **only same type** of data
- Structures can store **multiple different data types**
- Helps in organizing complex data clearly

Real-life example

A **Book** has:

- Title (string)
- Author (string)
- Price (float)
- Pages (int)

All these can be stored together using a structure.

---

2. Syntax of Structure Definition

```
struct structure_name {
   data_type member1;
   data_type member2;
   ...
};
```

Example

```
struct Book {
   char title[50];
   char author[50];
   float price;
   int pages;
```

};

- This defines a structure named **Book**
- Memory is **not allocated** until a variable is created

---

3. Declaring Structure Variables

Method 1: After structure definition

```
struct Book b1;
```

Method 2: Along with structure definition

```
struct Book {
    char title[50];
    char author[50];
    float price;
    int pages;
} b1, b2;
```

---

4. Accessing Structure Members

We use the **dot (.) operator** to access structure members.

Example

```
strcpy(b1.title, "C Programming");
strcpy(b1.author, "Dennis Ritchie");
b1.price = 450.50;
b1.pages = 300;
```

---

5. Complete Program: Structure Example

```
#include <stdio.h>
#include <string.h>

struct Book {
    char title[50];
    char author[50];
    float price;
```

```
    int pages;
};

int main() {
    struct Book b1;

    strcpy(b1.title, "C Programming");
    strcpy(b1.author, "Dennis Ritchie");
    b1.price = 450.50;
    b1.pages = 300;

    printf("Book Title: %s\n", b1.title);
    printf("Author: %s\n", b1.author);
    printf("Price: %.2f\n", b1.price);
    printf("Pages: %d\n", b1.pages);

    return 0;
}
```

---

6. Array of Structures

When we need to store **multiple objects** of the same type, we use an **array of structures**.

Syntax

```
struct Book books[5];
```

---

7. Activity: Create an Array of Books

Program: Array of Structures (Book)

```
#include <stdio.h>

struct Book {
    char title[50];
    char author[50];
```

```
    float price;
    int pages;
};

int main() {
    struct Book books[3];
    int i;

    for (i = 0; i < 3; i++) {
        printf("\nEnter details of Book %d\n", i + 1);

        printf("Title: ");
        scanf(" %[^\n]", books[i].title);

        printf("Author: ");
        scanf(" %[^\n]", books[i].author);

        printf("Price: ");
        scanf("%f", &books[i].price);

        printf("Pages: ");
        scanf("%d", &books[i].pages);
    }

    printf("\n--- Book Details ---\n");

    for (i = 0; i < 3; i++) {
        printf("\nBook %d\n", i + 1);
        printf("Title: %s\n", books[i].title);
        printf("Author: %s\n", books[i].author);
        printf("Price: %.2f\n", books[i].price);
```

```
        printf("Pages: %d\n", books[i].pages);
    }

    return 0;
}
```

---

8. Nested Structures

A **nested structure** is a structure inside another structure.

Example

A **Book** contains a **Publisher**.

```
struct Publisher {
    char name[50];
    char location[50];
};

struct Book {
    char title[50];
    float price;
    struct Publisher pub;
};
```

Accessing Nested Members

book.pub.name

book.pub.location

---

9. Union in C

What is a Union?

A **union** is similar to a structure but:

- All members **share the same memory**
- Only **one member can hold a value at a time**

Why use unions?

- Saves memory

- Used when only one value is needed at a time

---

10. Syntax of Union

```
union union_name {
   data_type member1;
   data_type member2;
};
```

Example

```
union Data {
   int i;
   float f;
   char c;
};
```

---

11. Difference Between Structure and Union

Feature	Structure	Union
Memory allocation	Separate memory for each member	Same memory shared
Access	All members at same time	Only one member
Size	Sum of all members	Size of largest member
Usage	Complex data	Memory optimization

---

12. Structure vs Union Example

```
#include <stdio.h>

union Data {
   int i;
   float f;
};

int main() {
```

```
    union Data d;

    d.i = 10;
    printf("Integer: %d\n", d.i);

    d.f = 3.14;
    printf("Float: %.2f\n", d.f);

    return 0;
}
```

---

13. Key Points to Remember

- Structures store **different data types together**
- Use . operator to access members
- Arrays of structures store **multiple records**
- Nested structures allow **complex data modeling**
- Unions save memory but allow **one value at a time**